

File: mL_algo_from_scratch/gradient_descent.py

Purpose: Source code implementation for study and analysis

Math Alert: This involves calculus! Derivatives are used to update weights.

```
"""Simple gradient descent helper.
```

```
This small class is intentionally simple and commented for clarity.
```

```
"""
```

```
import numpy as np
```

=== Classes ===

```
class GradientDescent:
```

Study Tip: Initialization sets up the state. Look for hyperparameters.

Logic Focus: Defined task specifically for '__init__'

```
    def __init__(self, lr=0.01, epochs=1000, tol=1e-8, verbose=False):
        self.lr = float(lr)
        self.epochs = int(epochs)
        self.tol = float(tol)
        self.verbose = bool(verbose)
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

Logic Focus: Defined task specifically for 'fit'

```
    def fit(self, X, y, grad_fn, theta0):
```

Math Alert: This involves calculus! Derivatives are used to update weights.

```
        """Run gradient descent using the user-provided gradient function.
```

Math Alert: This involves calculus! Derivatives are used to update weights.

```
        grad_fn(theta, X, y) should return a gradient vector same shape as theta.
        Returns final theta and an (optional) empty history list for compatibility.
        """
```

```
        theta = theta0.astype(float)
```

```
        history = []
```

```
        for i in range(self.epochs):
```

```
            grad = grad_fn(theta, X, y)
```

```
            old_theta = theta.copy()
```

```
            # update
```

```
            theta = theta - self.lr * grad
```

```
            # check for small change
```

```
            if np.linalg.norm(theta - old_theta) < self.tol:
```

```
                if self.verbose:
```

```
                    print("converged at epoch", i)
```

```
                break
```

```
        return theta, history
```